MTRC

MODERN INSTITUTE OF TECHNOLOGY AND RESEARCH CENTRE

**Name of Faculty**: Sanjeev Khatri

**Subject & Subject Code**: **Semester:** OP using C++ - 3CS4-04 **Session:** 2023-24(Odd Sem.)

**Unit No.:** 5 **Date of Submission:**

| Lect.No. | Topics Covered | Page No. | Total pages |
|---|---|---|---|
| 1 | Template Classes, Template Functions | 1-6 | 6 |
| 2 | Standard Template Library : Fundamental Idea about String | 7-10 | 4 |
| 3 | Iterators, Hashes | 11-14 | 4 |
| 4 | Iostreams and Other Types | 15-17 | 3 |
| 5 | Namespaces : User Defined Namespaces | 18-20 | 3 |
| 6 | Namespaces provided by Library | 21-22 | 2 |
| 7 | Object Oriented Design, Design and Programming, Role of Classes | 23-25 | 3 |

**References Book:**


1. Object Oriented Programming with C++ - E Balagurusamy
2. C++ : The Complete Reference – Herbert Schlitz
3. Object Oriented Programming in Turbo C++ - Robert Lafore.
4. C++ How to Program – Paul Dietel


05/10/2023

**FACULTY** **HOD** **DEAN ACADEMICS**

# UNIT-5

# LECTURE - 1

# Template

Templae is an advanced feature of C++. It make it possible to use one function or class to handle many different data types. The template concept can be used in two different ways: with functions and with classes. By using templates, it is possible to create generic functions and classes.

## Generic Functions or Function Templates

A generic function defines a general set of operations that will be applied to various types of data. The type of data that the function will operate upon is passed to it as a parameter. If we want to write a function that returns the absolute value of two numbers, then we need to write a function. But when we need to get absolute value for integer, float, double, we have to write three different functions. But, there is a way to write such a function just once, and have it work for many different data types and that is possible by the use of **Function Template** or **Generic Function**.

A generic function is created using the keyword **template**. It is used to create a template (or framework) that describes what a function will do.

The general form of a **template function** definition is -

```
template <class Ttype> ret-type func-name(parameter list)
{
        // body of function
}
```

Here, Ttype is a placeholder name for a data type used by the function. This name may be used within the function definition.

**Example-1**-

```
#include <iostream>
using namespace std;
```



MTRC

```
template <class T> T abs(T n) //function template
{
   return (n < 0) ? -n : n;
}

int main()
{
   int int1 = 7;
   int int2 = -8;
   long lon1 = 90000L;
   long lon2 = -50000L;
   double dub1 = 6.64;
   double dub2 = -3.25;
   //calls instantiate functions
   cout << "\nabs(" << int1 << ")=" << abs(int1); //abs(int)
   cout << "\nabs(" << int2 << ")=" << abs(int2); //abs(int)
   cout << "\nabs(" << lon1 << ")=" << abs(lon1); //abs(long)
   cout << "\nabs(" << lon2 << ")=" << abs(lon2); //abs(long)
   cout << "\nabs(" << dub1 << ")=" << abs(dub1); //abs(double)
   cout << "\nabs(" << dub2 << ")=" << abs(dub2); //abs(double)
   cout << endl;
   return 0;
}
```

**Output**:

```
abs(7)=7
```

MITRC

abs(-8)=8

abs(90000)=90000

abs(-50000)=50000

abs(6.64)=6.64

abs(-3.25)=3.25

As we can see, the abs() function now works with all three of the data types (int, long, and double) that we use as arguments.

## Function Templates with Multiple Arguments

We can define more than one generic data type in the template statement by using a comma-separated list.

**Example**-

```
#include <iostream>

using namespace std;


template <class t1, class t2> void func(t1 x, t2 y)

{

   cout << x << ' ' << y << '\n';

}


int main()

{

   func(55, "Welcome");

   func(23.7, 23L);

   return 0;

}
```

**Output**-

55 Welcome

23.7 23

MITRC

## Applying Generic Functions

Generic functions are one of C++'s most useful features. They can be applied to all types of situations. If we have a function that defines a generalizable algorithm, we can make it into a template function.

## Generic Classes or Class Templates

The template concept can be extended to classes. Class templates are generally used for data storage (container) classes.

Function templates allow us to create generic functions that have the same bodies but can take different data types as parameters. Likewise, in some situations classes are similar and we want to perform very similar operations with them. If we need to create several similar classes, we might consider developing a class template to generate a class in which at least one type is generic or parameterized.

The general form of a **generic class** declaration is -

```
template <class Ttype> class class-name
{
  ...
}
```

Here, Ttype is the placeholder type name, which will be specified when a class is instantiated. If necessary, you can define more than one generic data type using a comma-separated list.

**Example**-

```
#include <iostream>
using namespace std;
const int MAX = 100; //size of array

template <class Type> class Test
{
   private:
      Type st[MAX]; //Test: array of any type
```

MITRC

```
    int top; //number of top of Test
  public:
    Test() //constructor
    { top = -1; }

    void push(Type var) //put number on Test
    { st[++top] = var; }

    Type pop() //take number off Test
    { return st[top--]; }
};

int main()
{
  Test<float> t1; //t1 is object of class Test<float>
  t1.push(2222.2F); //push 3 floats, pop 3 floats
  t1.push(3333.3F);
  t1.push(4444.4F);
  cout << "1: " << t1.pop() << endl;
  cout << "2: " << t1.pop() << endl;
  cout << "3: " << t1.pop() << endl <<"\n";
  Test<long> t2; //t2 is object of class Test<long>
  t2.push(456456456L); //push 3 longs, pop 3 longs
  t2.push(234235234L);
  t2.push(789789789L);
  cout << "1: " << t2.pop() << endl;
  cout << "2: " << t2.pop() << endl;
  cout << "3: " << t2.pop() << endl;
```

MITRC

```
    return 0;
}
```

**Output**-

1. 4444.4
2. 3333.3
3. 2222.2

1. 789789789
2. 234234234
3. 456456456

Here the class Test is presented as a template class. The approach is similar to that used in function templates. The template keyword and class Test signal that the entire class will be a template.

Class templates differ from function templates in the way they are instantiated. To create an actual function from a function template, we call it using arguments of a specific type. Classes, however, are instantiated by defining an object using the template argument.

A template class can have more than one generic data type. Simply declare all the data types required by the class in a comma-separated list within the template specification.

## The Power of Templates

- Templates help us to achieve one of the most elusive goals in programming: the creation of reusable code.
- With the use of template classes we can create frameworks that can be applied over and over again to a variety of programming situations.
- Generic functions and classes provide a powerful tool that is, we can use them to amplify our programming efforts.

MITRC

# LECTURE - 2

## Standard Template Library

Standard Template Library (STL) provides general-purpose, templatized classes and functions that implement many popular and commonly used algorithms and data structures, including support for vectors, lists, queues, and stacks. It also defines various routines that access them. Because the STL is constructed from template classes, the algorithms and data structures can be applied to nearly any type of data.

At the core of the standard template library are three foundational items: containers, algorithms, and iterators. These items work in conjunction with one another to provide off-the-shelf solutions to a variety of programming problems.

### Containers

Containers are objects that hold other objects, and there are several different types. For example, the vector class defines a dynamic array, deque creates a double-ended queue, and list provides a linear list. These containers are called sequence containers because in STL terminology, a sequence is a linear list. the STL also defines associative containers, which allow efficient retrieval of values based on keys. For example, a map provides access to values with unique keys. Thus, a map stores a key/value pair and allows a value to be retrieved given its key.

### Algorithms

Algorithms act on containers. They provide the means by which you will manipulate the contents of containers. Their capabilities include initialization, sorting, searching, and transforming the contents of containers.

### Iterators

Iterators are objects that act, more or less, like pointers. They give you the ability to cycle through the contents of a container in much the same way that you would use a pointer to cycle through an array.

MITRC

Each container has defined for it an allocator. Allocators manage memory allocation for a container. The default allocator is an object of class allocator, but we can define our own allocators if needed by specialized applications.

## Why use Standard Template Library (STL)

- STL offers an assortment of containers.
- STL publicizes the time and storage complexity of its containers.
- STL containers grow and shrink in size automatically.
- STL provides built-in algorithms for processing containers.
- STL provides iterators that make the containers and algorithms flexible and efficient.
- STL is extensible which means that users can add new containers and new algorithms such that:
  - STL algorithms can process STL containers as well as user defined containers
  - User defined algorithms can process STL containers as well user defined containers.

## Fundamental Idea about String

The C++ standard library provides a common implementation of a string class abstraction named string. The string class is a specialization of a more general template class called basic_string. The string class is very large, with many constructors and member functions. The string class is not part of the STL, but has implemented many STL features. **string** can be treated as a STL container of char.

**A commonly used example using string class-**

```
#include <iostream>

#include <string>

using namespace std;


int main()

{

        string str = “Hello World!”;

        cout << str << endl;

        return 0;

}
```

MITRC

The string class supports several constructors. The prototypes for three of its most commonly used ones are shown here:

string( );

string(const char *str);

string(const string &str);

The first form creates an empty string object. The second creates a string object from the null-terminated string pointed to by str. This form provides a conversion from nullterminated strings to string objects. The third form creates a string from another string.

To use the string type, we simply include its header file.
#include <string>

To create a string, we can do-

string str = “some string text”;

or

string str(“some string text);

or

string str1(5, ‘s’);

string str2 = str1;

## string length

The length of string is returned by its size() operation. For example-

string str = “some text”;

cout << str.size(); //showing length of str.

## string concatenation

Concatenation means combine two or more texts. Concatenating one string to another is done by the ‘+’ operator. For example-

MITRC

```
string s1 = “Hello ”;
string s2 = “World”;
string c_str = s1 + sr2;
```

## string comparison

To check if two strings are equal, we use the ‘==‘ operator. For example-

```
string s1 = “Hello ”;
string s2 = “World”;
if (s1 == s2) //compairing two string;
```

## string assignment

To assign one string to another, we use the “=“ operator. For example-

```
string s1 = “Hello ”;
string s2 = “World”;

s1 = s2;
```

MITRC

# LECTURE - 3

# Iterators, Hashes

## Iterators

An iterator is a generalization (or perhaps more precisely, an abstraction) of a pointer. Iterators are handled in our program like pointers, and they implement the standard pointer operators. They give us the ability to cycle through the contents of a container in much the same way that we would use a pointer to cycle through an array. In general, an iterator that has greater access capabilities can be used in place of one that has lesser capabilities.

## The Basic Iterator Types

There are five types of iterators:

| Iterator | Access Allowed |
|---|---|
| Random Access | Store and retrieve values. Elements may be accessed randomly. |
| Bidirectional | Store and retrieve values. Forward and backward moving. |
| Forward | Store and retrieve values. Forward moving only. |
| Input | Retrieve but not store values. Forward moving only. |
| Output | Store but not retrieve values. Forward moving only. |

## Input Iterator

An input iterator is an iterator used to access the elements from the container, but it does not modify the value of a container.

Operators used for an input iterator are:

Increment operator(++)

Equal operator(==)

Not equal operator(!=)

Dereference operator(*)

## Output Iterator

An output iterator is an iterator used to modify the value of a container, but it does not read the value from a container. Therefore, we can say that an output iterator is a write-only iterator.



MITRC

Operators used for an output iterator are:

Increment operator(++)

Assignment operator(=)

## Forward Iterator

A forward iterator is an iterator used to read and write to a container. It is a multi-pass iterator.

Operators used for a Forward iterator are:

Increment operator(++)

Assignment operator(=)

Equal operator(=)

Not equal operator(!=)

## Bidirectional iterator

A bidirectional iterator is an iterator supports all the features of a forward iterator plus it adds one more feature, i.e., decrement operator(--). We can move backward by decrementing an iterator.

Operators used for a Bidirectional iterator are:

Increment operator(++)

Assignment operator(=)

Equal operator(=)

Not equal operator(!=)

Decrement operator(--)

## Random Access Iterator

A Random Access iterator is an iterator provides random access of an element at an arbitrary location. It has all the features of a bidirectional iterator plus it adds one more feature, i.e., pointer addition and pointer subtraction to provide random access to an element.

MITRC

**Example**-

```
#include <iostream>
#include<vector>
#include<iterator>
using namespace std;

int main()
{
   vector<int> vctr{4,5,6,7,8};

   vector<int>::iterator itratr;

   for(int i=0;i<5;i++)
   {
      cout<<vctr[i]<<" ";
   }

   cout<<'\n';
   for(itratr=vctr.begin();itratr!=vctr.end();itratr++)
   {
      cout<<*itratr<<" ";
   }

   vctr.push_back(10);
   cout<<'\n';

   for(int i=0; i<6; i++)
   {
```

MITRC

```
    cout<<vctr[i]<<" ";
  }

  cout<<'\n';
  for(itratr=vctr.begin();itratr!=vctr.end();itratr++)
  {
    cout<<*itratr<<" ";
  }
   return 0;
}
```

**Output**-

4 5 6 7 8

4 5 6 7 8

4 5 6 7 8 10

4 5 6 7 8 10

## Hashes

The hash class is default constructible, which means that one can construct this object without any arguments or initialization values. It is used to get the hash value of the argument that is being passed to it.

**Syntax**:

template <class T> struct hash;

**Syntax to create object**:

hash<class template> object-name

MITRC

# LECTURE - 4

# Iostreams

## Stream Classes

A stream is a general name given to a flow of data. In C++ a stream is represented by an object of a particular class. Different streams are used to represent different kinds of data flow.

## Advantages of Streams

- There are no such formatting characters in streams, since each object already knows how to display itself.
- We can overload existing operators and functions, such as the insertion (<<) and extraction (>>) operators, to work with classes that we create.
- They are the best way to write data to files, and also to format data in memory for later use in text input/output windows and other GUI elements.

## The ios Class

The ios class is the grand daddy of all the stream classes, and contains the majority of the features we need to operate C++ streams. The three most important features are the formatting flags, the error-status flags, and the file operation mode.

## Formatting Flags

Formatting flags are a set of enum definitions in ios. They act as on/off switches that specify choices for various aspects of input and output format and operation

## Manipulators

Manipulators are formatting instructions inserted directly into a stream. For example-endl-which sends a newline to the stream and flushes it.

## Functions

The ios class contains a number of functions that we can use to set the formatting flags and perform other tasks. For example-width-which is used to get or set width of current field.



MITRC

## The istream Class

The istream class, which is derived from ios, performs input-specific activities, or extraction. It's easy to confuse extraction and the related output activity, insertion.

## The ostream Class

The ostream class handles output or insertion activities.

## The iostream Class

The iostream class, which is derived from both istream and ostream, acts only as a base class from which other classes can be derived. It has no functions of its own (except constructors and destructors). Classes derived from iostream can perform both input and output.

## The get() and put() Functions

The classes istream and ostream define two member functions get() and put() to handle the single character input/output operations respectively.

There are two types of get() functions. We can use both get(char*) and get(void) prototypes to fetch a character including the blank space, tab and the newline character. The get(char*) version assigns the input character to its argument and the get(void) version returns the input character.

**Example**-

```
#include<iostream>
using namespace std;
int main()
{
   char c;
   cin.get(c); //read a character
   cout.put(c); //display the character on screen cin.get (c ) ;
   return 0;
}
```

MITRC

## The getline () and write () Functions

We can read and display a line of text more efficiently using the line-oriented input/output functions getline() and write(). The getline() function reads a whole line of text that ends with a newline character.

**Example**-

```
#include<iostream>

using namespace std;

int main()

{

   char c[10];

   cin.getline(c, 10); //read characters

   cout.write(c, 10); //display the characters on screen;

   return 0;

}
```

## Formatted I/O

The C++ I/O system allows you to format I/O operations. For example, you can set a field width, specify a number base, or determine how many digits after the decimal point will be displayed.

MITRC

# LECTURE - 5

## Namespaces

Named entities, such as variables, functions, and compound types need to be declared before being used in C++. The purpose of namespaces is to localize the names of identifiers to avoid name collisions. Namespaces allow us to group named entities that otherwise would have global scope into narrower scopes, giving them namespace scope. This allows organizing the elements of programs into different logical scopes referred to by names.

The syntax to declare a namespace is:

```
namespace identifier
{
  named_entities
}
```

identifier is any valid identifier and named_entities is the set of variables, types and functions that are included within the namespace.

```
namespace namspac
{
        int r, s;
}
```

In this case, the variables r and b are normal variables declared within a namespace called namspac.

**Example**-

```
#include <iostream>
using namespace std;
namespace func
{
   int value()
   {
      return 5;
```

MITRC

```
   }
}
namespace useofpi
{
   const double pi = 3.1416;
   double value()
   {
      return 2*pi;
   }
}
int main ()
{
   cout << func::value() << '\n';
   cout << useofpi::value() << '\n';
   cout << useofpi::pi << '\n';
   return 0;
}
```

output:

8

12.5664

3.1416

**using**

The keyword using introduces a name into the current declarative region (such as a block), thus avoiding the need to qualify the name.

**Example**-

```
#include <iostream>
using namespace std;
namespace namspac1
```



MITRC

```
{
   int a = 3;
   int b = 4;
}
namespace namspac2
{
   double a = 3.1416;
   double b = 2.7183;
}
int main ()
{
   using namspac1::a;
   using namspac2::b;
   cout << a << '\n';
   cout << b << '\n';
   cout << namspac1::b << '\n';
   cout << namspac2::a << '\n';
   return 0;
}
```

**Output**:

```
3
2.7183
4
3.1416
```

## Namespace aliasing

Existing namespaces can be aliased with new names, with the following syntax:

```
namespace new_name = current_name;
```

MITRC

# LECTURE - 6

## Namespaces Provided by Library

**There are three main namespaces**:

### std

The ISO C++ standards specify that "all library entities are defined within namespace std." This includes namespaces nested within namespace std, such as namespace std::chrono.

### abi

Specified by the C++ ABI. This ABI specifies several type and function APIs supplemental to those required by the ISO C++ Standard, but necessary for interoperability.

### __gnu_

Indicating one of several GNU extensions. Choices include __gnu_cxx, __gnu_debug, __gnu_parallel, and __gnu_pbds.

### The std namespace

Standard C++ defines its entire library in its own namespace called std. All the entities (variables, types, constants, and functions) of the standard C++ library are declared within the std namespace. As we use - using namespace std;

This introduces direct visibility of all the names of the std namespace into the code. This is done in these tutorials to facilitate comprehension and shorten the length of the examples, but many programmers prefer to qualify each of the elements of the standard library used in their programs. The using directive allows all the names in a namespace to be used without the namespace-name as an explicit qualifier.

For example, when we write-

```
cout << "Hello world!";
```

It is common to instead see:

```
std::cout << "Hello world!";
```



MITRC

This causes the std namespace to be brought into the current namespace, which gives us direct access to the names of the functions and classes defined within the library without having to qualify each one with std::.

MITRC

# LECTURE - 7

# Object Oriented Design

The software development procedures integrate the methods and tools together and enable rational and timely development of software system. There exist several software development paradigms, each using a different set of methods and tools. The development of a successful system depends not only on the use of the appropriate methods and techniques but also on the developer's commitment to the objectives of the system.

## Procedure-Oriented Paradigm

Software development is usually characterized by a series of stages depicting the various tasks involved in the development process.

**Classic Life Cycle**-

Problem definition-Analysis-Design-Coding-Testing-Maintenance

The classic life cycle is based on an underlying mode, commonly referred to as the **water-fall** model. The Classic Software life cycle is most widely used for the procedure-oriented development.

## Development Tools

Many tools are used in the analysis and design of the system. The development tools available may be classified as first, second, third, fourth and fifth generation tools. These tools are often packaged as RAD (Rapid Application Development) or CASE (Computer-Aided Software Engineering) tools.

## Object-Oriented Paradigm

The object-oriented paradigm draws heavily on the general systems theory as a conceptual background. A system can be viewed as a collection of entities that interact together to accomplish certain objectives. Entities may represent physical objects such as equipment and people and abstract concepts such as data files and functions. In object-oriented analysis, the entities are called objects.

MITRC

## Development Tools for Object-Oriented Design Paradigm

1. Classes and Objects
2. Instances of Objects
3. Message Communication between Objects
4. Inheritance Relationship
5. Classification Relationship
6. Composition Relationship
7. Class Diagram
8. Object Diagram
9. Use Case Diagram
10. Activity Diagram
11. Sequence Diagram
12. State Diagram
13. Component Diagram
14. Deployment Diagram
15. Package Diagram

## Steps in Object-Oriented Analysis

The object-oriented analysis (OOA) approach consists of the following steps:

1. Understanding the problem.
2. Drawing the specification of requirements of the user and the software
3. Identifying the objects and their attributes
4. Identifying the services that each object is expected to provide (interface).
5. Establishing inter-connection (collaborations) between the objects in terms of services required and services rendered.

## Steps in Object-Oriented Design

Design is concerned with the mapping of objects in the problem space into objects in the solution space and creating an overall structure (architectural model) and computational models of the system. This stage normally uses the bottom-up approach to build the structure of the system and the top-down functional decomposition approach to design the class member functions that provide services.

The object-oriented design (OOD) approach may involve the following steps:

1. Review of objects created in the analysis phase
2. Specification of class dependencies
3. Organization of class hierarchies
4. Design of Classes



MTRC

5. Design of member functions
6. Design of driver program

## Role of Classes

A C++ class combines data and methods for manipulating the data into one. Classes also determine the forms of objects. The data and methods contained in a class are known as class members. A class is a user-defined data type. To access the class members, we use an instance of the class. A class is a blueprint for an object. Reusability of classes from the previous designs, classification of the objects into subsystems and determination of appropriate protocols are some of the considerations of the design stage.

MITRC